

Working with models

Models, which represent back-end data sources or services, are the heart of LoopBack. They come with full-featured REST APIs. *Built-in models* are created implicitly as per the application template; users add *custom models* with the help of generation wizards.

Let's add a model using a generator for this app, which represents some environmental parameters like temperature, humidity, pressure, etc.

```
lb model
? Enter the model name: Weather
? Select the datasource to attach weather to: db (memory)
? Select model's base class : PersistedModel
? Expose weather via the REST API? Yes
? Custom plural form (used to build REST URL): Weather
? Common model or server only? common
```

You can identify the generated file for the created model as *weather.json* in the *common/models* sub-directory. The default plural form is mostly suitable for model names, e.g., *'Devices'* is applicable if the model name is *'Device'*. But for the current model, we'll use the custom plural form, as *'Weathers'* is not a suitable plural form. Let's add a few properties to this model, say *device-id* as the string, and *temperature*, *humidity*, *pressure* as the number with a suitable choice for *Required fields* and *Default values*. You can also add more properties after completing the wizard using the following command or by manually editing *weather.json*:

```
lb property
```

Connecting with data sources

Model instances (property values) are stored under the in-memory database, by default. You can opt for better persistency support with suitable connectors for popular SQL and NoSQL based databases like MySQL, MongoDB, etc.

Let's use the following steps to create a new data source and attach the model to it:

```
lb datasource
? Enter the datasource name: mydb
? Select the connector for mydb: MySQL
? Connection String url to override other settings:
? host: localhost
? port: 3306
? user: root
? password: ****
? database: loopback
? Install loopback-connector-mysql@*2.2 (Y/n)Yes
```

This will install *loopback-connector-mysql* in *node_modules* and generate JSON content for the data source in

server/datasources.json. This file can be edited for further changes once the wizard completes its tasks. To attach this data source to our *'weather'* model, edit *server/model-config.json* as follows, replacing *db* with *mydb*:

```
"Weather": {
  "dataSource": "mydb",
  "public": true
}
```

LoopBack comes with support for preparing schema with applicable tables or collections in chosen databases; so, let's add a script in *server/boot*, say *prepare-schema.js* with the following content:

```
module.exports = function(app) {
  app.datasources.mydb.autoupdate('Weather', function(err) {
    if (err) throw err;
    console.log('Schema created for Weather model: \n');
  });
}
```

Create the database *loopback* in MySQL and launch the server. You can observe that any updation in model instances is reflected in the chosen database under the table with the same name as the model. You can follow similar steps for other databases like MongoDB, with suitable connectors.

Exploring REST APIs

Each generated model comes with full-featured REST APIs with 'create, read, update and delete' (CRUD) operational support on model instances. You can explore these APIs using the Swagger UI available at *hostname:3000/explorer*. Let's perform the following REST operations to add a new instance:

```
POST /api/Weather
{
  "device-id": "rpi123"
  "temperature": 18,
  "humidity": 72,
  "pressure": 1060
}
```

And to retrieve all model instances, use the following command:

```
GET /api/Weather
```

You can also add filters to retrieve specific instances in a particular order. Examples are given below.

To list instances with a specific device ID, type:
{*"where"* : {*"device-id"*: *"rpi3"*}}